

《VHDL-EG01》

实验指导书



前 言

- 1、实验前必须认真预习实验指导书及实验内容，明确实验目的、步骤、原理、回答实验教师的提问，回答不合要求者，须重新预习，才能进行实验。
- 2、对规定实验外确属需要的内容，可先提出实验原理和方法，经教师或实验技术人员同意后，方可进行实验。
- 3、做实验时必须严格遵守实验室的规章制度和仪器设备的操作规程，服从教师和实验技术人员的指导。
- 4、爱护仪器设备，节约使用材料，使用前详细检查，使用后要整理就位，发现丢失或损坏应立即报告，未经许可不得动用与本实验无关的仪器设备及其它物品，不准将任何实验室物品带出室外。
- 5、实验时必须注意安全，防止人身和设备事故的发生，若发生事故应立即切断电源，及时向指导教师报告，并保持现场，不得自行处理，待指导教师查明原因并排除故障后，方可继续实验。
- 6、进入实验室后应保持安静，不得高声喧哗和打闹，不准抽烟，不准随地吐痰，不准乱抛纸屑杂物，要保持实验室和仪器设备的整齐清洁。
- 7、实验完毕后，经实验室工作人员检查仪器设备、工具、材料及实验记录后方可离开。
- 8、实验后要认真完成实验报告，包括分析结果、处理数据、绘制曲线及图表等。对不合格要求的实验报告应退回重做。
- 9、对违反实验规章制度和操作规程、擅自动用与本实验无关的仪器设备、私自拆卸仪器而造成事故和损失的肇事者必须写出书面检查，视情节轻重和认识程度按规定处理。
- 10、在进入实验室前，务必搞好个人卫生，不得将脏物带入室内，有净化要求的实验室，进室必须换拖鞋。

目 录

实验一 VIVADO软件的使用	4
实验二 计数器设计	10
实验三 分频器设计	14
实验四 序列检测器的设计	18
实验五 流水灯设计	22
实验六 数控分频器设计	27
实验七 VGA显示控制电路设计	30
实验八 串口控制器设计	36

实验一 VIVADO软件的使用

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

- 1 熟悉 Vivado 软件创建工程和编辑设计文件的方法；
- 2 熟悉创建工程文件的方法；
- 3 熟悉 VHDL Test Bench 文件编写；
- 4 会用 VHDL 语言设计数据选择器；
- 5 会用 VHDL 语言设计半加器和全加器。

二、实验原理

多路选择器又叫“多路开关”。数据选择器在地址码(选择控制)电位的控制下，从几个数据中选择一个数据送到公共的输出端。

三、实验内容

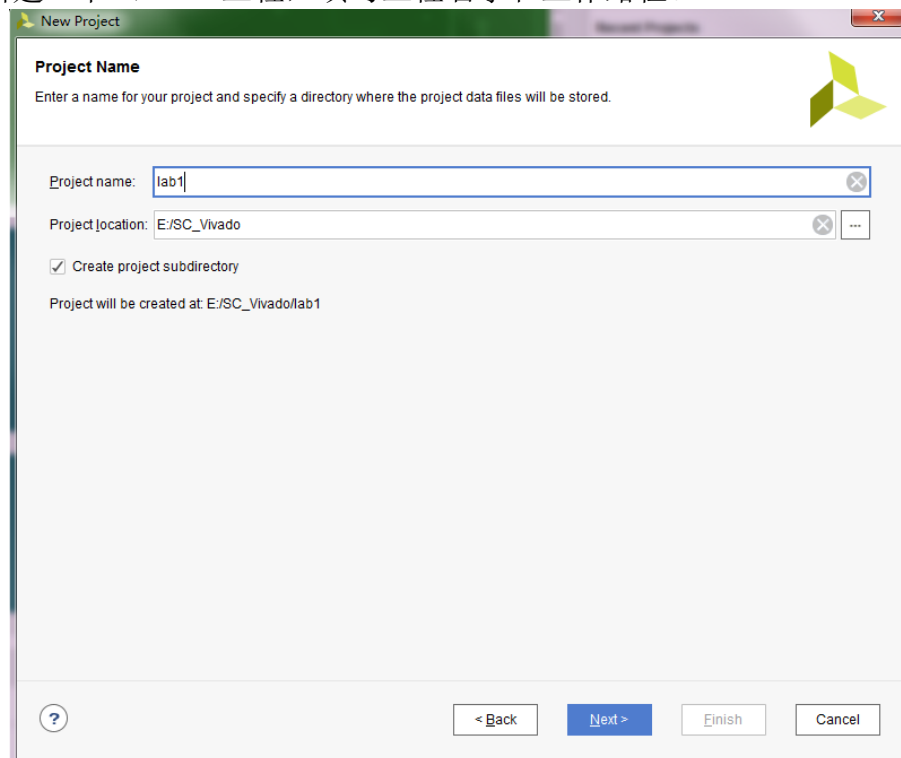
- 1 创建新工程，完成多路选择器的 VHDL 描述；
- 2 检查程序是否正确；
- 3 建立 Test Bench 文件，对设计进行仿真，分析仿真结果。
- 4 用 VHDL 设计全加器，完成仿真，分析仿真结果。

四、实验步骤

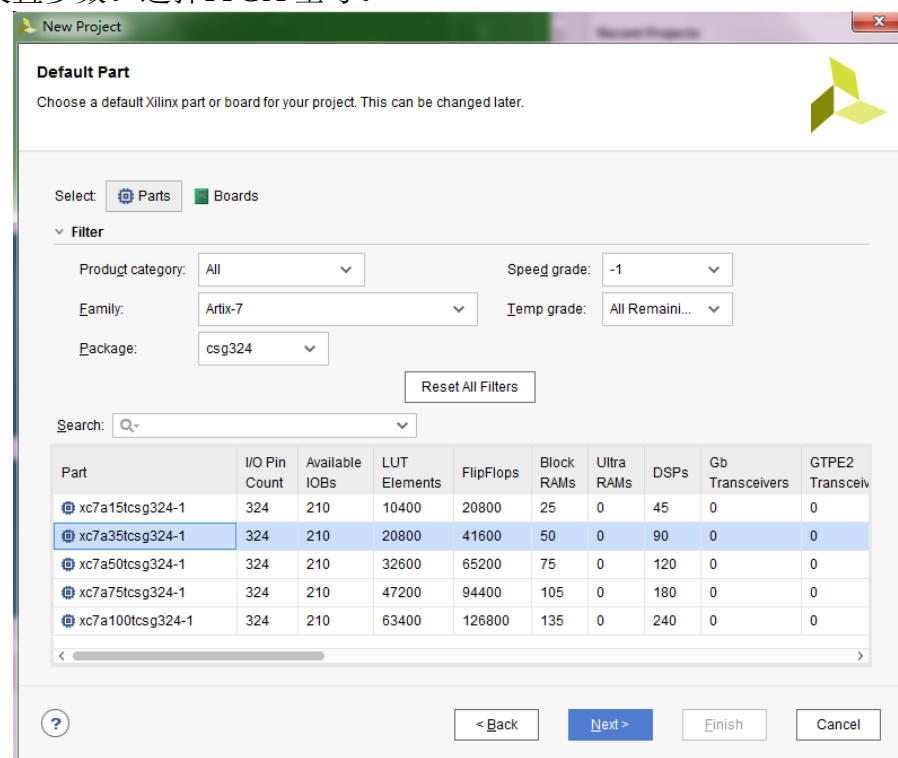
- 1、双击桌面图标，进入软件界面。



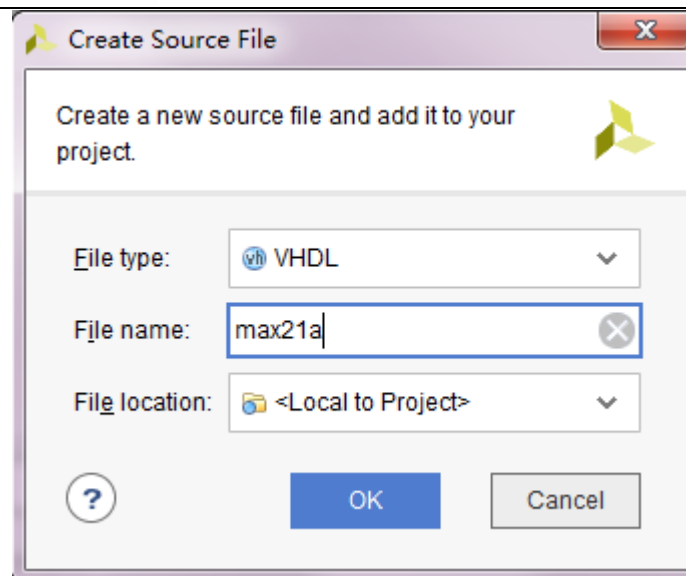
2、新建一个 VHDL 工程，填写工程名字和工作路径。



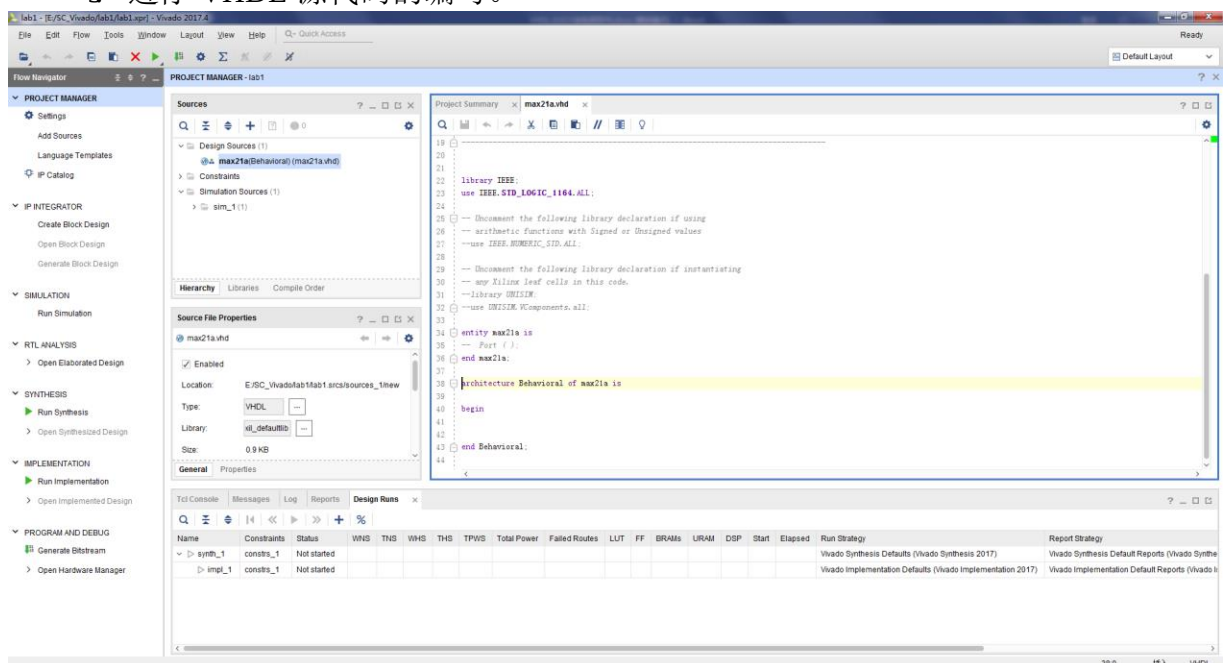
3、设置参数。选择FPGA 型号。



4、新建工程文件文件，建立 VHDL 模块。

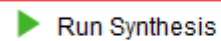


5 进行 VHDL 源代码的编写。



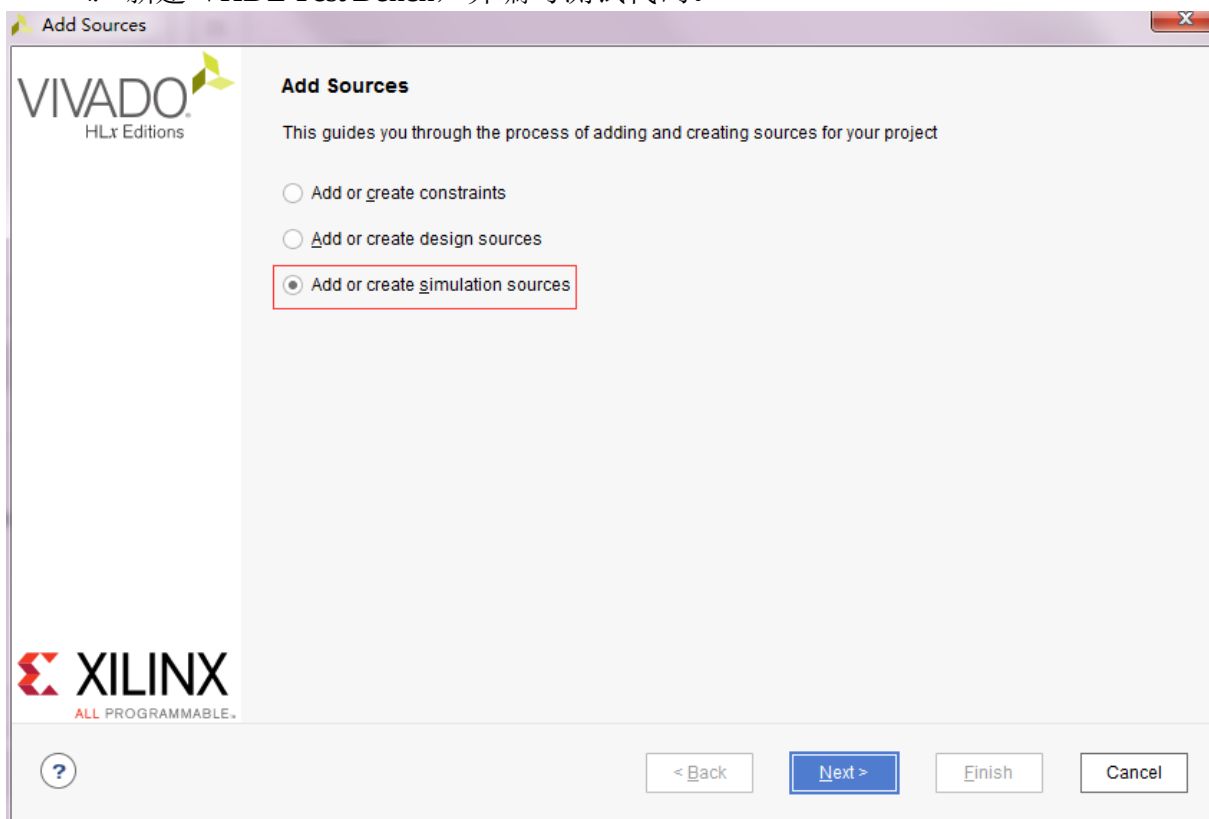
6、检查语法错误。

SYNTHESIS

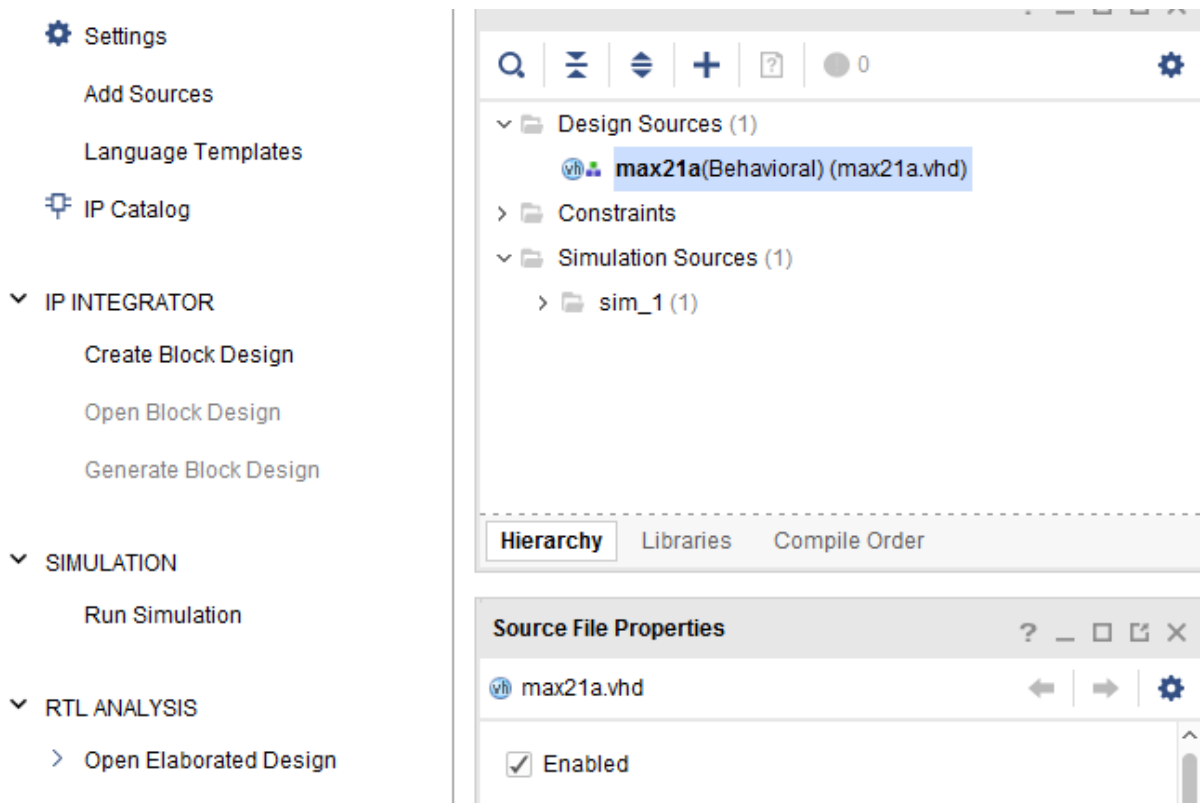
 Run Synthesis

> Open Synthesized Design

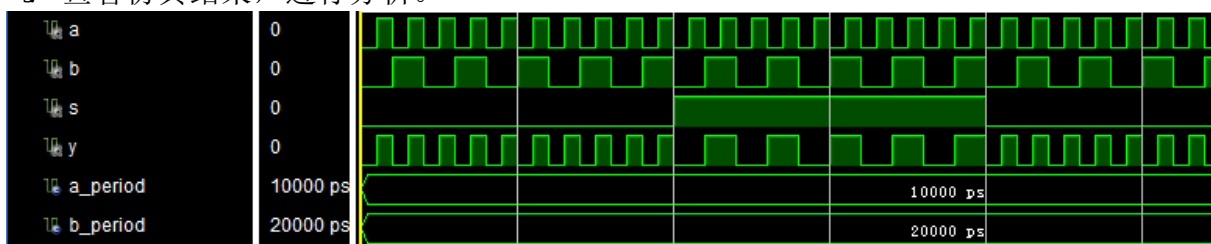
7 新建 VHDL Test Bench，并编写测试代码。



8 检查语法错误，对模块进行仿真



9 查看仿真结果，进行分析。



五、练习

1. 新建一个工程，设计一个四选一多路选择器，并进行功能仿真。
2. 新建一个工程，设计一个半加器，利用设计的半加器设计一个全加器，并进行功能仿真。

六、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、仿真结果及分析。
实验报告内容为四选一多路选择器模块和全加器模块。

附件：二选一多路选择器代码

示例一：

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux21a is
    Port ( a : in  STD_LOGIC;
          b : in  STD_LOGIC;
          s : in  STD_LOGIC;
          y : out STD_LOGIC);
end mux21a;

architecture Behavioral of mux21a is
begin
    y <= a when s = '0'
        else b ;
end Behavioral;
```

附件：二选一多路选择器 Test Bench 代码

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY tb_mux21a IS
END tb_mux21a;

ARCHITECTURE behavior OF tb_mux21a IS

    COMPONENT mux21a
    PORT(
        a : IN  std_logic;
        b : IN  std_logic;
        s : IN  std_logic;
        y : OUT std_logic
    );
    END COMPONENT;

    --Inputs
    signal a : std_logic := '0';
    signal b : std_logic := '0';
    signal s : std_logic := '0';

    --Outputs
    signal y : std_logic;

    constant a_period : time := 10 ns;
    constant b_period : time := 20 ns;

BEGIN
    -- Instantiate the Unit Under Test (UUT)
    uut: mux21a PORT MAP (
        a => a,
        b => b,
        s => s,
        y => y
    );

    -- Clock process definitions
    a_process :process
    begin
        a <= '0';
        wait for a_period/2;
        a <= '1';
        wait for a_period/2;
    end process;

    b_process :process
    begin
        b <= '0';
        wait for b_period/2;
        b <= '1';
        wait for b_period/2;
    end process;

    -- Stimulus process
    stim_proc: process
    begin
        -- hold reset state for 100 ns.
        wait for 100 ns;
        s <= '1';
        wait for a_period*10;
        s <= '0' ;
        -- insert stimulus here
        wait;
    end process;

END;
```

实验二 计数器设计

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

1. 掌握同步计数器的逻辑功能及使用方法；
2. 学会用 VHDL 语言设计同步计数器。

二、实验原理

同步时序电路输出状态的变化与时钟同步，并且每个状态都是稳定状态，在任意时刻，几个输入变量可以同时变化。

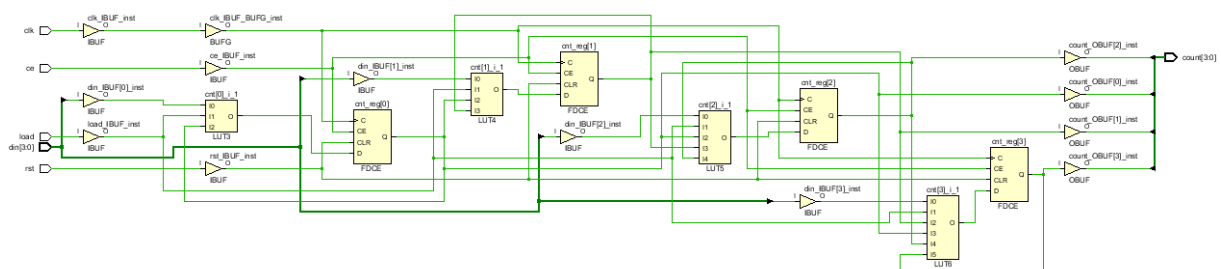
三、实验内容

1. 新建一个同步计数器工程；
2. 用 VHDL 语言编写同步计数器源程序；
3. 创建 Test bench，对计数器进行功能仿真。
4. 查看计数器的 RTL 结构。

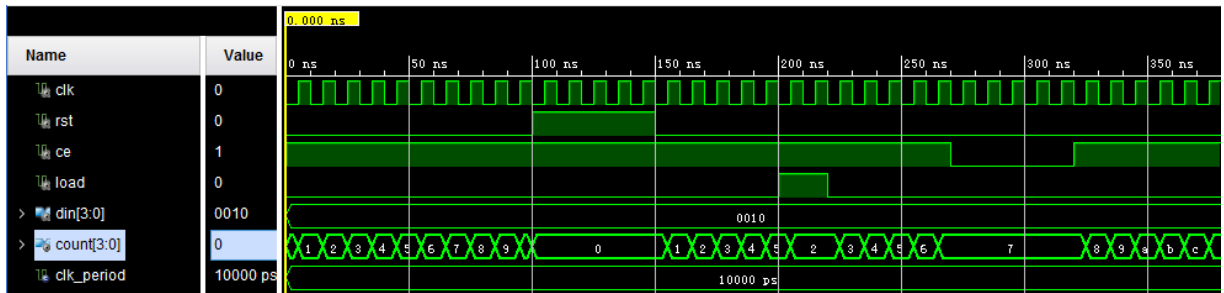
四、实验步骤

- 1、新建一个工程，使用 VHDL 设计一个同步计数器，计数器具有异步复位和同步预置数功能。
- 2、检查语法错误，综合后，查看 RTL 结构。

同步计数器的 RTL 结构如下。



3. 建立 Test bench 文件，对计数器进行仿真。对仿真结果进行分析。



五、练习

1. 修改同步计数器，设计一个同步 10 进制计数器(计数值最大为 9)，查看 RTL 结构，完成仿真，分析仿真结果。

六、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、仿真结果及分析，RTL 结构。实验报告内容为同步 10 进制计数器。

附件：同步计数器的 VHDL 描述

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.ALL;
entity counter4b is
    port (
        clk: in STD_LOGIC;
        rst: in STD_LOGIC;
        ce: in STD_LOGIC;
        load: in STD_LOGIC;
        din: in STD_LOGIC_VECTOR (3 downto 0);
        count: out STD_LOGIC_VECTOR (3 downto 0));
end counter4b;
architecture Behavioral of counter4b is
    signal cnt: std_logic_vector(3 downto 0):=(others =>'0');
begin
    process(clk,rst)
    begin
        if rst='1' then
            cnt <= (others => '0');
        else if (clk'event and clk='1') then
            if ce='1' then
                if load = '1' then
                    cnt <= din;
                else
                    cnt <= cnt + 1;
                end if;
            end if;
        end if;
    end process;
    count <= cnt;
end Behavioral;
```

附件：同步计数器的 Test Bench 文件

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
] entity tb_counter4b is
] end tb_counter4b;

] architecture Behavioral of tb_counter4b is
]   component counter4b
]     port( clk : in STD_LOGIC;
]           rst : in STD_LOGIC;
]           ce : in STD_LOGIC;
]           load : in STD_LOGIC;
]           din : in STD_LOGIC_VECTOR (3 downto 0);
]           count : out STD_LOGIC_VECTOR (3 downto 0)
]     );
]   end component;
]   signal clk : std_logic := '0';
]   signal rst : std_logic := '0';
]   signal ce : std_logic := '0';
]   signal load : std_logic := '0';
]   signal din : std_logic_vector(3 downto 0);
]   signal count : std_logic_vector(3 downto 0);

]   constant clk_period : time :=10 ns;
]   begin
]     uut: counter4b port map(
]       clk => clk,
]       rst => rst,
]       ce => ce,
]       load => load,
]       din => din,
]       count => count
]     );

]   clk_process :process
]   begin
]     clk <= '0';
]     wait for clk_period/2;
]     clk <= '1';
]     wait for clk_period/2;
]   end process;
]   stim_proc: process
]   begin
]     ce <= '1';
]     rst <= '0';
]     load <= '0';
]     wait for 100 ns;
]     rst <= '1';
]     wait for clk_period*5;
]     rst <= '0';
]     wait for clk_period*5;
]     load <= '1';
]     wait for clk_period*2;
]     load <= '0';
]     wait for clk_period*5;
]     ce <= '0';
]     wait for clk_period*5;
]     ce <= '1';
]     wait;
]   end process;
] end;
```

实验三 分频器设计

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

1. 掌握分频器的逻辑功能和使用方法；
2. 会用 VHDL 设计一个偶数分频器；
3. 会用 VHDL 设计一个奇数分频器；

二、实验原理

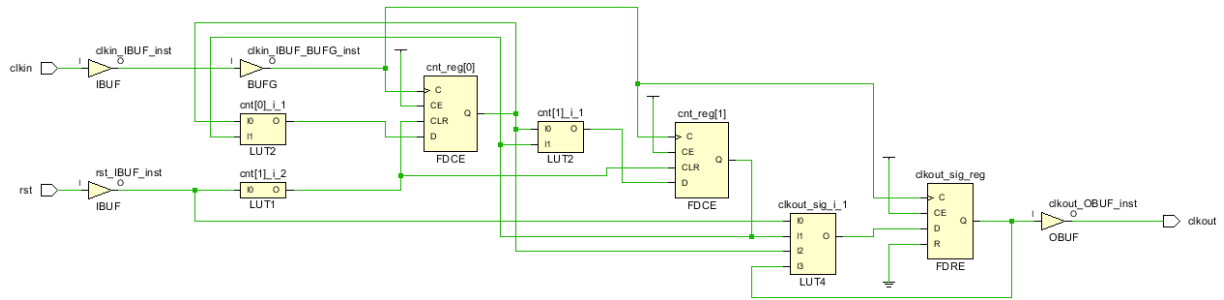
分频器实际上是计数器的应用，如：六分频器可以设计成一个三进制计数器，计完三个数后反相再计三个数，即实现了六分频。

三、实验内容

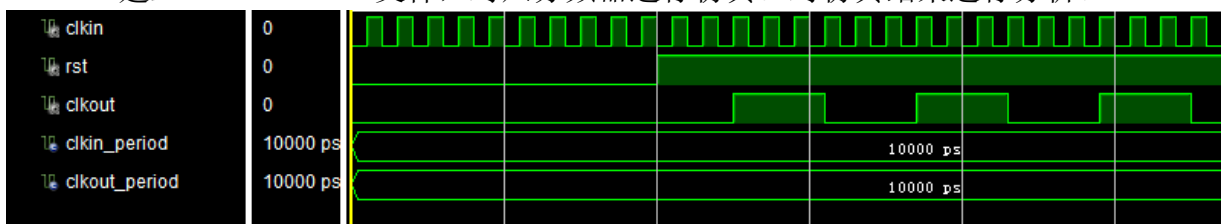
- 1 创建新工程，完成六分频器的 VHDL 描述；
- 2 检查程序是否正确，查看 RTL 结构图；
- 3 建立 Test Bench 文件，对设计进行仿真，分析仿真结果。
- 4 用 VHDL 设计五分频器，完成仿真，分析仿真结果。

四、实验步骤

- 1、新建一个工程，使用 VHDL 设计一个六分频器，分频后的信号具有 50% 的占空比。
- 2、检查语法错误，综合后，查看 RTL 结构。



3. 建立 Test bench 文件，对六分频器进行仿真。对仿真结果进行分析。



五、练习

修改分频器，分别设计一个七分频器和八分频器，查看 RTL 结构，完成仿真，分析仿真结果。

六、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、仿真结果及分析，RTL 结构。实验报告内容为七分频器和八分频器。

附件：六分频器的 VHDL 描述

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
entity div6 is
    Port(
        clkkin: in STD_LOGIC;
        rst: in STD_LOGIC;
        clkout: out STD_LOGIC
    );
end div6;
architecture Behavioral of div6 is
    signal clkout_sig: std_logic:= '0';
begin
    clkout <= clkout_sig;
    process(clkkin, rst)
        variable cnt: integer range 0 to 3:=0;
    begin
        if rst = '0' then
            cnt :=0;
        else if rising_edge(clkkin) then
            cnt:=cnt + 1;
            if cnt=3 then
                cnt :=0;
                clkout_sig <= not clkout_sig;
            end if;
        end if;
    end if;
end process;
end Behavioral;
```


附件：六分频器的 Test Bench 文件

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_div6 is
end tb_div6;

architecture Behavioral of tb_div6 is
    component div6
    port( clkkin : in STD_LOGIC;
          rst : in STD_LOGIC;
          clkout : out STD_LOGIC
        );
    end component;
    signal clkkin : std_logic := '0';
    signal rst : std_logic := '0';
    signal clkout : std_logic;
    constant clkkin_period : time := 10 ns;
    constant clkout_period : time := 10 ns;
    begin
        uut: div6 port map(
            clkkin => clkkin,
            rst => rst,
            clkout => clkout
        );
        clkkin_process : process
        begin
            clkkin <= '0';
            wait for clkkin_period/2;
            clkkin <= '1';
            wait for clkkin_period/2;
        end process;
        stim_proc: process
        begin
            rst <= '0';
            wait for 100 ns;
            rst <= '1';
            wait for clkkin_period*10;
            wait;
        end process;
    end;
```

实验四 序列检测器的设计

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

用状态机实现序列检测器的设计，并对其进行仿真。

二、实验原理

序列检测器可用于检测一组或多组由二进制码组成的脉冲序列信号，当序列检测器连续收到一组串行二进制码后，如果这组码与检测器中预先设定的码相同，则输出 1，否则输出 0。由于这种检测方法的关键在于正确码收到必须是连续的，要求检测器必须记住上次的正确码，直到在连续的检测中收到的每个码都与预置数对应的码相同。检测过程中，任意一位不相等都将回到初始状态重新开始检测。

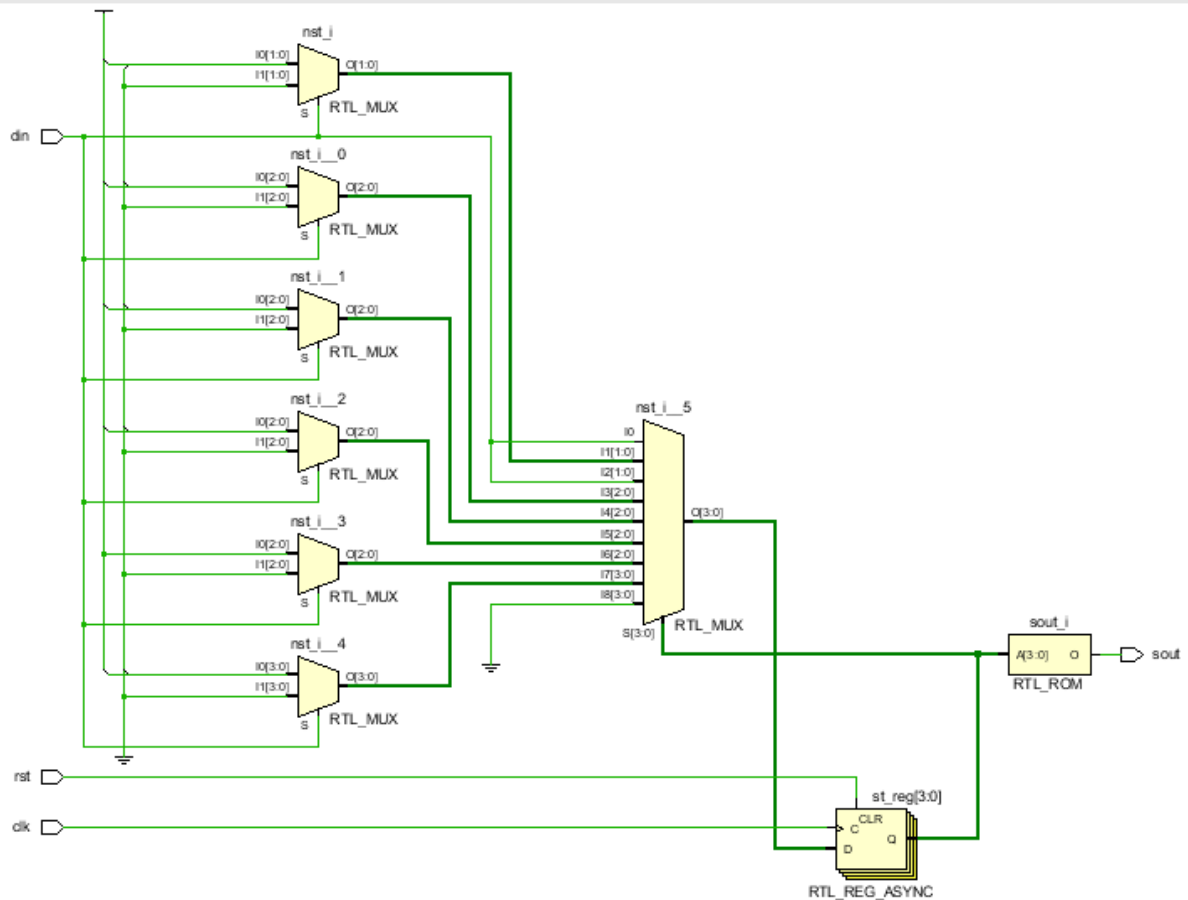
三、实验内容

1. 新建一个序列检测器工程；
2. 用 VHDL 语言编写序列检测器源程序；
3. 创建 Test bench，对序列检测器进行功能仿真。
4. 查看序列检测器的 RTL 结构。

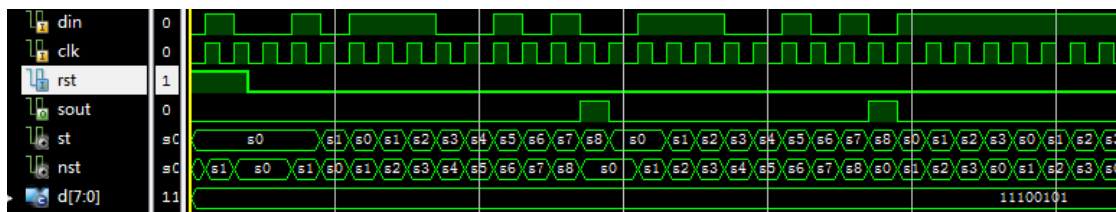
四、实验步骤

- 1、新建一个工程，使用 VHDL 设计一个序列检测器，检测到预置的序列后，输出为 1，否则为 0。

2、检查语法错误，综合后，查看 RTL 结构。



3. 建立 Test bench 文件，对序列检测器进行仿真。对仿真结果进行分析。



五、练习

1. 修改序列器，设计一个“10100111”序列检测器，完成仿真，分析仿真结果。

六、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、仿真结果及分析。

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity schk is
port(
    din,clk,rst: in std_logic;
    sout:        out std_logic
    );
end schk;

architecture Behavioral of schk is
    --待检测序列
    constant D:std_logic_vector(7 downto 0):="11100101";
    type states is (s0,s1,s2,s3,s4,s5,s6,s7,s8,s9);
    signal st,nst:states:=s0;
begin

    process(clk,rst)
    begin
        if rst='1' then
            st <=s0;
        elsif clk'event and clk='1' then
            st <= nst;
        end if;
    end process;

    process(st,din)
    begin
        case st is
            when s0 =>
                if din = D(7) then
                    nst <= s1;
                else
                    nst <= s0;
                end if;
            when s1 =>
                if din = D(6) then
                    nst <= s2;
                else
                    nst <= s0;
                end if;
        end case;
    end process;
```

```
        when s2 =>
            if din = D(5) then
                nst <= s3;
            else
                nst <= s0;
            end if;
        when s3 =>
            if din = D(4) then
                nst <= s4;
            else
                nst <= s0;
            end if;
        when s4 =>
            if din = D(3) then
                nst <= s5;
            else
                nst <= s0;
            end if;
        when s5 =>
            if din = D(2) then
                nst <= s6;
            else
                nst <= s0;
            end if;
        when s6 =>
            if din = D(1) then
                nst <= s7;
            else
                nst <= s0;
            end if;
        when s7 =>
            if din = D(0) then
                nst <= s8;
            else
                nst <= s0;
            end if;
        when others => nst <=s0;
    end case;
end process;

sout <= '1' when st = s8
        else '0';
end Behavioral;
```

附件：序列检测器 Test Bench

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY tb_schk IS
END tb_schk;

ARCHITECTURE behavior OF tb_schk IS
    COMPONENT schk
    PORT (
        din : IN  std_logic;
        clk : IN  std_logic;
        rst : IN  std_logic;
        sout : OUT std_logic
    );
    END COMPONENT;
--Inputs
signal din : std_logic := '0';
signal clk : std_logic := '0';
signal rst : std_logic := '0';

--Outputs
signal sout : std_logic;

-- Clock period definitions
constant clk_period : time := 10 ns;
constant len: integer range 0 to 30:=24;
signal data:std_logic_vector(len downto 0):="1001011100101001110010101";
BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: schk PORT MAP (
        din => din,
        clk => clk,
        rst => rst,
        sout => sout
    );

    -- Clock process definitions
    clk_process :process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;
    -- Stimulus process
    stim_proc: process
    begin
        rst <= '1';
        wait for 20 ns;
        rst <= '0';
        wait for clk_period*10;
        wait;
    end process;

    process(clk)
    begin
        if rising_edge(clk) then
            din <= data(len);
            data(len downto 1) <= data(len-1 downto 0);
        end if;
    end process;
END;
```

实验五 流水灯设计

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

- 6 熟悉 Vivado 软件创建工程和编辑设计文件的方法；
- 7 熟悉创建工程文件的方法；
- 8 会用 VHDL 语言设计流水灯；

二、实验原理

建立时钟中间信号进行计数，预置操作数，LED灯由左向右依次点亮。

三、实验内容

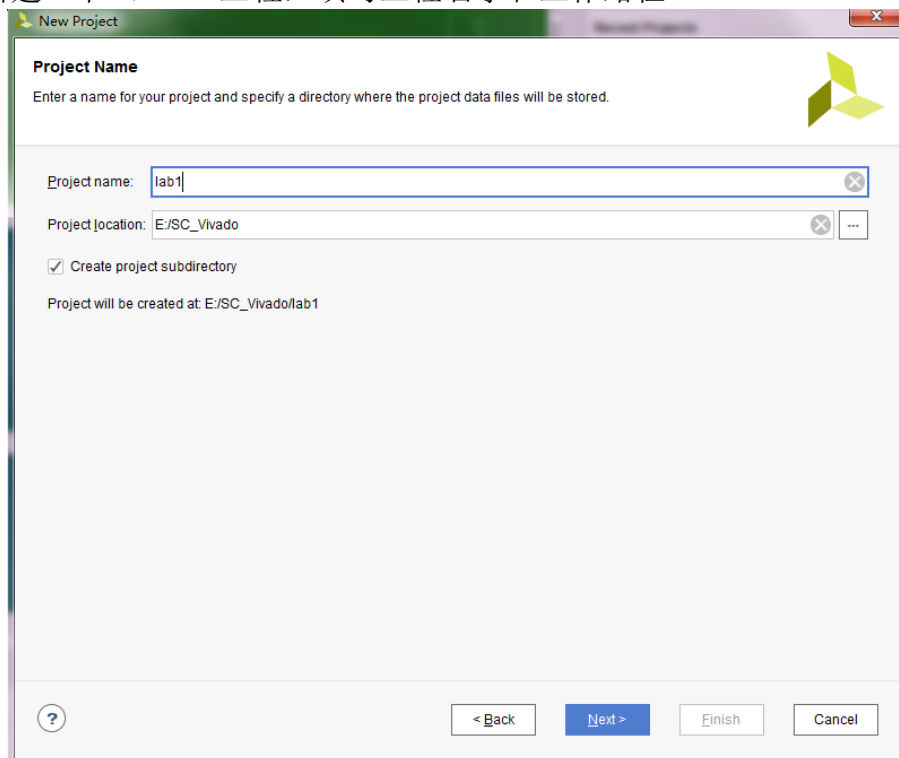
- 6 创建新工程，完成流水灯的 VHDL 描述；
- 7 检查程序是否正确；
- 8 用 VHDL 设计流水灯，观察分析实验结果。

四、实验步骤

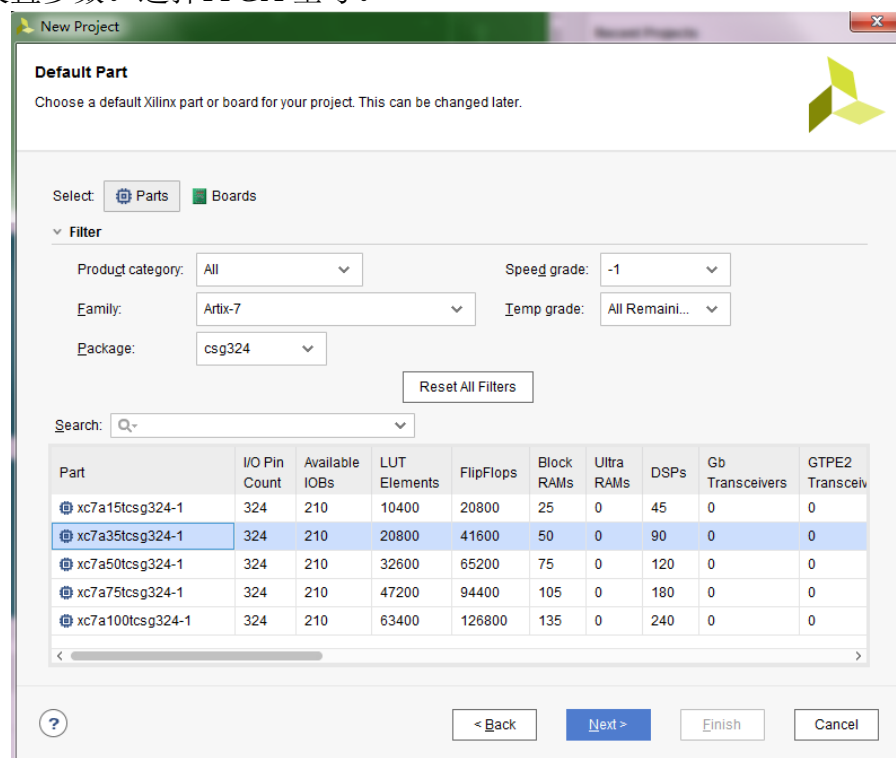
- 1、双击桌面图标，进入软件界面。



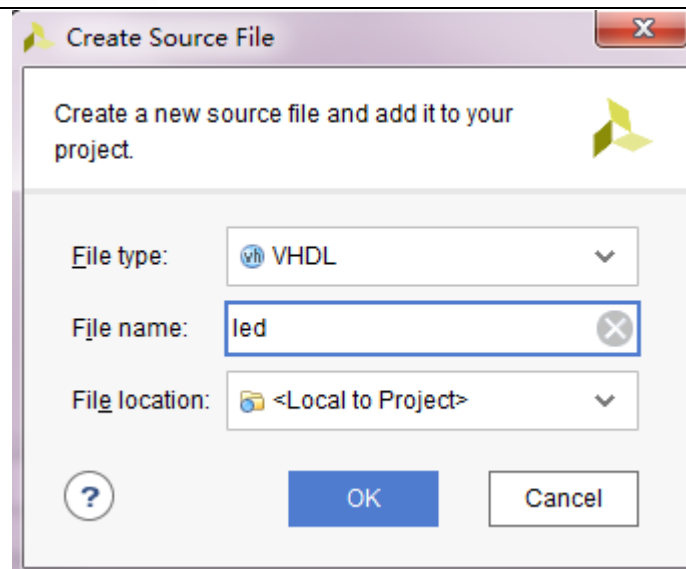
2、新建一个 VHDL 工程，填写工程名字和工作路径。



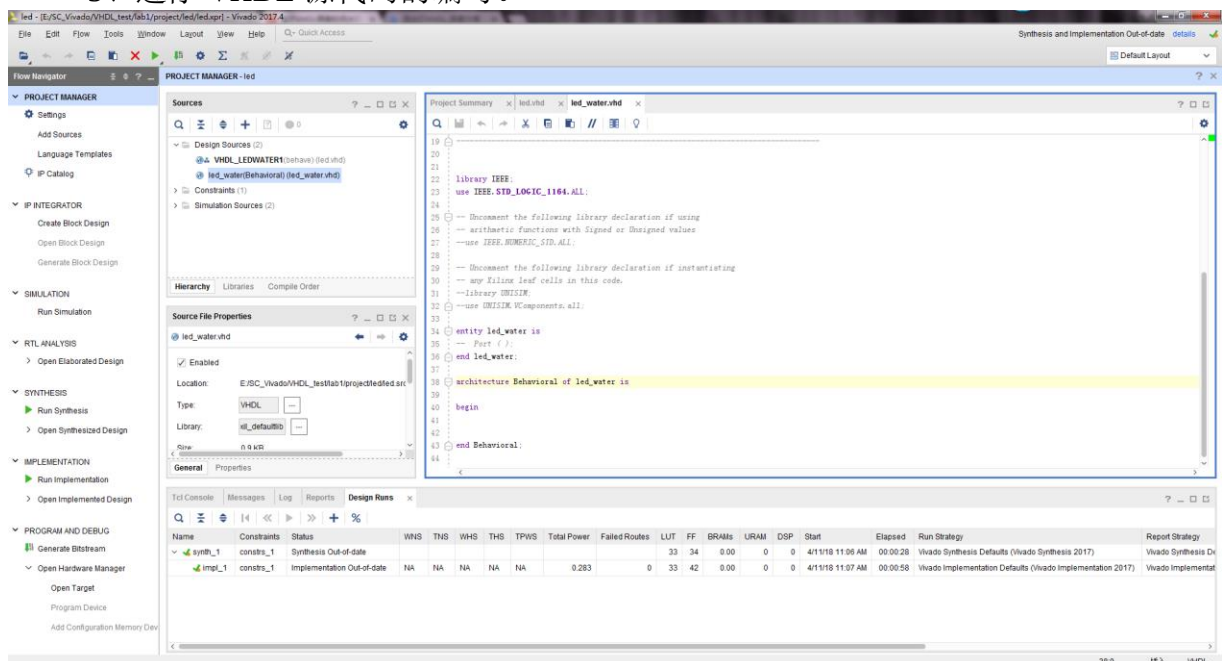
3、设置参数。选择FPGA 型号。



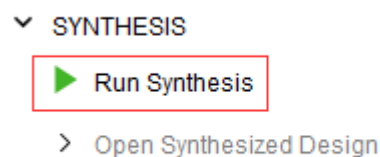
4、新建工程文件文件，建立 VHDL 模块。



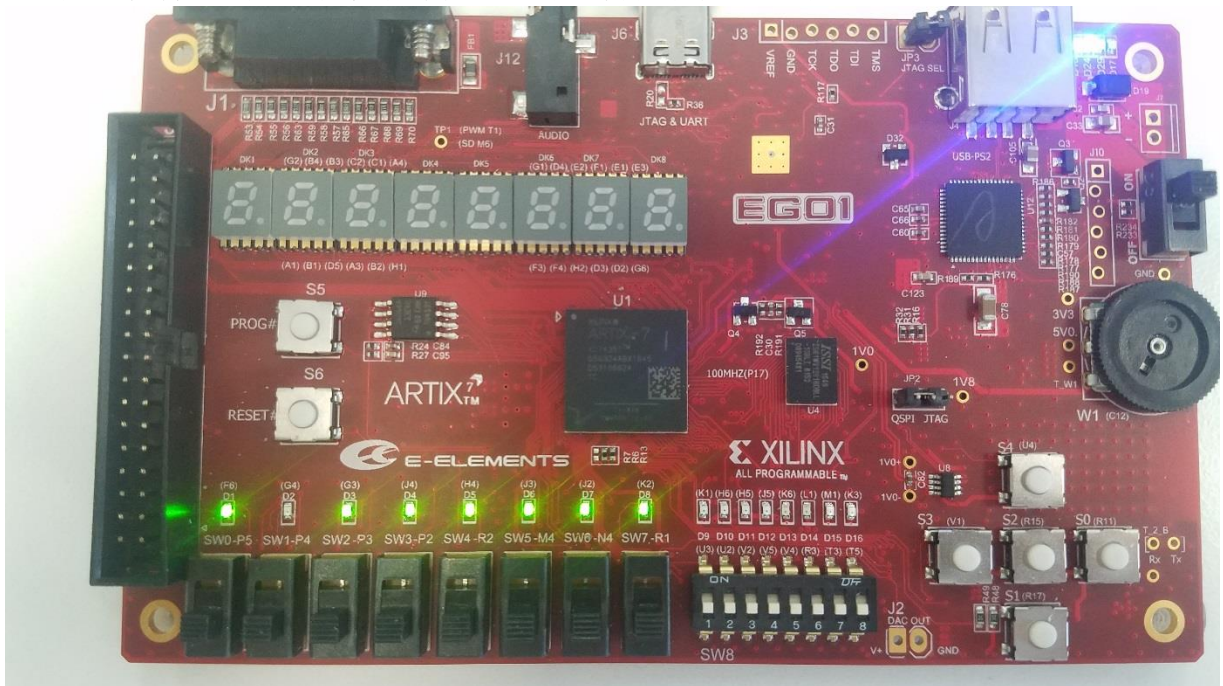
5、进行 VHDL 源代码的编写。



6、检查语法错误。



10 分配管脚下载至开发板观察实验结果。并分析实验结果。



五、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、实验结果分析。实验报告内容为流水灯设计。

附件：流水灯代码示例：

```
library IEEE;
use IEEE.std_logic_1164.all;

entity VHDL_LEDWATER1 is
    port (
        Clk      : in  STD_LOGIC;           --创建时钟端口，连接开发板PIN23
        Rst       : in  STD_LOGIC;           --创建复位端口，连接开发板PIN116
        Output    : out BIT_VECTOR(7 downto 0) --创建输出端口，对应8个LED。
                                                --其左侧必须为BIT_VECTOR类型
    );
end VHDL_LEDWATER1;

architecture behave of VHDL_LEDWATER1 is
    signal Clk1 : STD_LOGIC;               --建立中间时钟信号
begin
    P1:process(Clk)
        variable count : INTEGER range 0 to 25000000 := 0;
        variable count1: STD_LOGIC := '1';
    begin
        if(Rst = '0') then
            count := 0;
        elsif(Clk'event and Clk = '1') then
            count := count + 1;
            if(count = 25000000) then --这里使用=, 而不是>=, 可以防止产生比较器, 节省硬件资源
                count := 0;
                count1 := not count1;
            end if;
        end if;
        Clk1 <= count1;
    end process P1;
    P2:process(Clk1)
        variable temp : BIT_VECTOR(7 downto 0) := "11111110"; --注意左操作数类型
    begin
        if(Clk1'event and Clk1 = '1') then
            temp := (temp rol 1);
        end if;
        Output <= temp;
    end process P2;
end architecture;
```

实验六 数控分频器设计

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

3. 学习数控分频器的设计、分析方法；
4. 学会用 VHDL 语言设计数控分频器。

二、实验原理

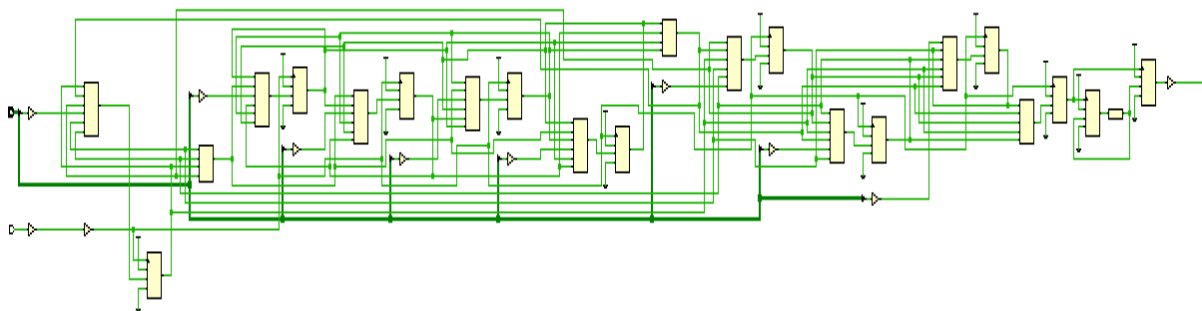
在当输入端给定不同输入数据时，将对输入的时钟信号有不同的分频比。计数值并行预置加法计数器，计数溢出位与预置数加载输入信号相接。

三、实验内容

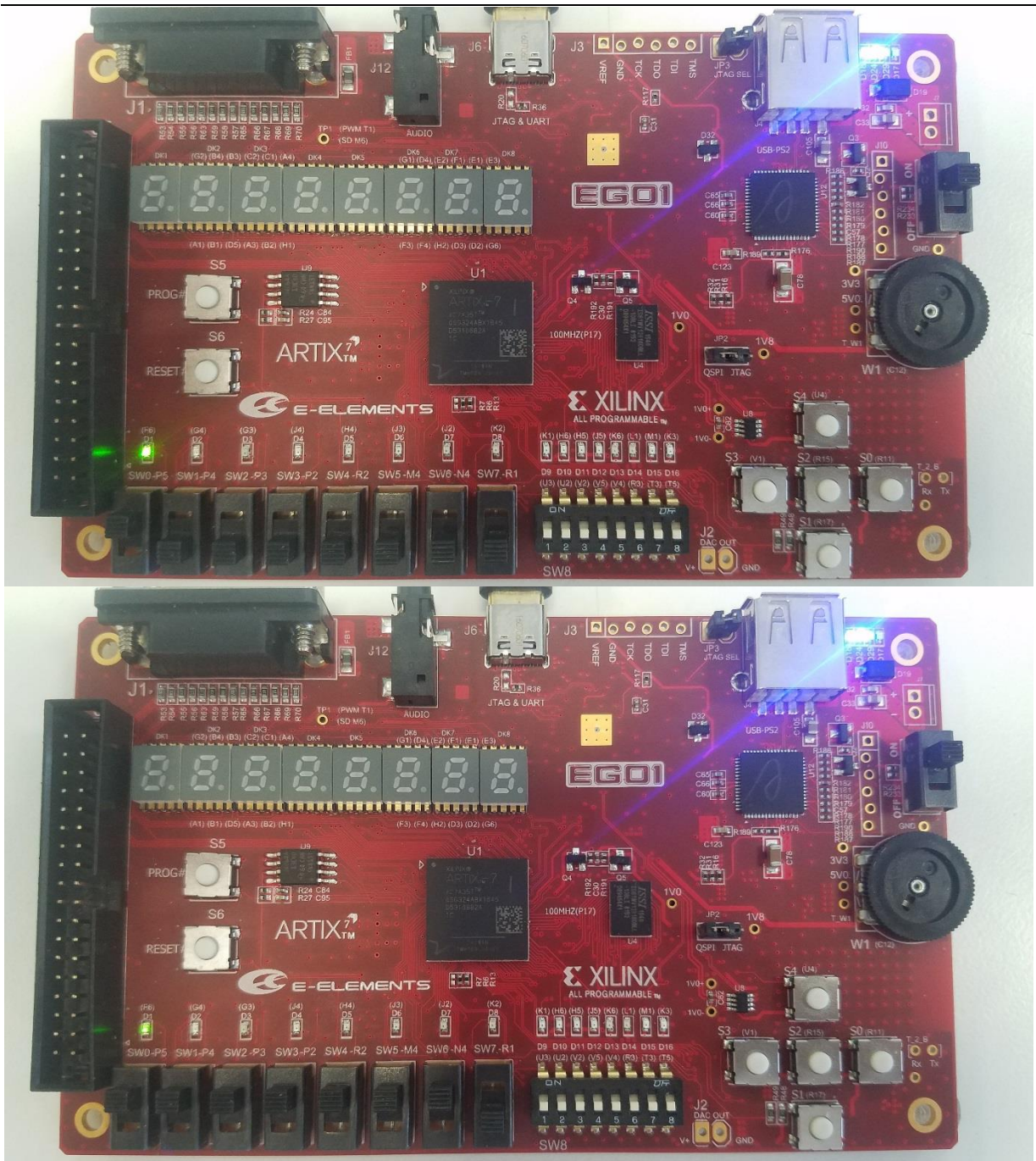
5. 新建一个同步计数器工程；
6. 用 VHDL 语言编写同步计数器源程序；
7. 查看分析数控分频器实验结果。

四、实验步骤

- 1、新建一个工程，使用 VHDL 设计一个并行预置加法计数器和溢出位部分。
- 2、检查语法错误，综合后，查看 RTL 结构。



3. 分配管脚下载至开发板观察实验结果。并分析实验结果。



五、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、实验结果分析，RTL 结构。实验报告内容为数控分频器设计。

附件：数控分频器的VHDL 描述

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
] ENTITY counter_div IS
    PORT (
        CLK : IN STD_LOGIC;
        D   : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        FOUT : out std_logic
    );
] END ENTITY counter_div;
] ARCHITECTURE ONE OF counter_div IS
    SIGNAL FULL: STD_LOGIC;
    BEGIN
]     P_REG: PROCESS(CLK)
        VARIABLE CNT18: STD_LOGIC_VECTOR(7 DOWNTO 0);
        BEGIN
]         IF CLK'EVENT AND CLK='1' THEN
]             IF CNT18="11111111" THEN
                    CNT18:=D;
                    FULL<='1';
                    ELSE CNT18:=CNT18+1;
                    FULL<='0';
]             END IF;
]         END IF;
]     END PROCESS P_REG;
]
P_DIV: PROCESS(FULL)
    VARIABLE CNT2: STD_LOGIC;
    BEGIN
        IF FULL'EVENT AND FULL='1' THEN
            CNT2:= NOT CNT2;
            IF CNT2='1' THEN
                FOUT<='1';
            ELSE
                FOUT<='0';
            END IF;
        END IF;
    END PROCESS P_DIV;
END ARCHITECTURE ONE;

```

实验七 VGA显示控制电路设计

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

4. 掌握VGA显示控制的时序和接口标准；
5. 学会VGA显示控制电路的设计方法；

二、实验原理

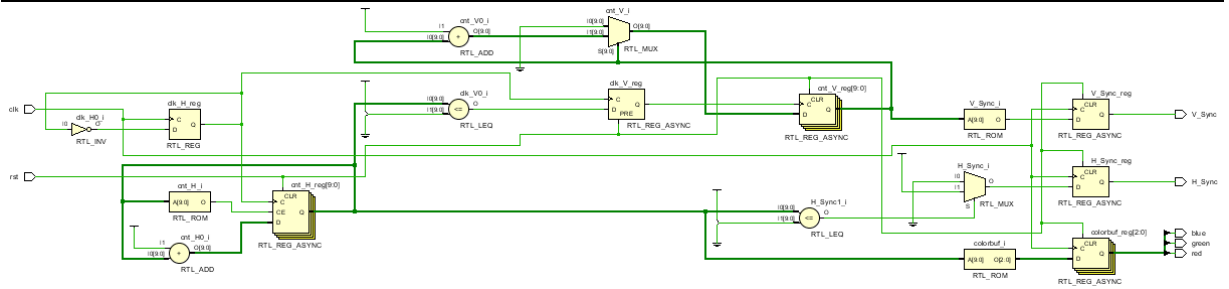
VGA端口15根线，其中5根为行同步信号输入，场同步信号输入，红色信号输入，绿色信号输入，蓝色信号输入。VGA扫描从上到下，从左到右，本实验一帧扫描640*480个像素点。行同步，场同步为负极性。

三、实验内容

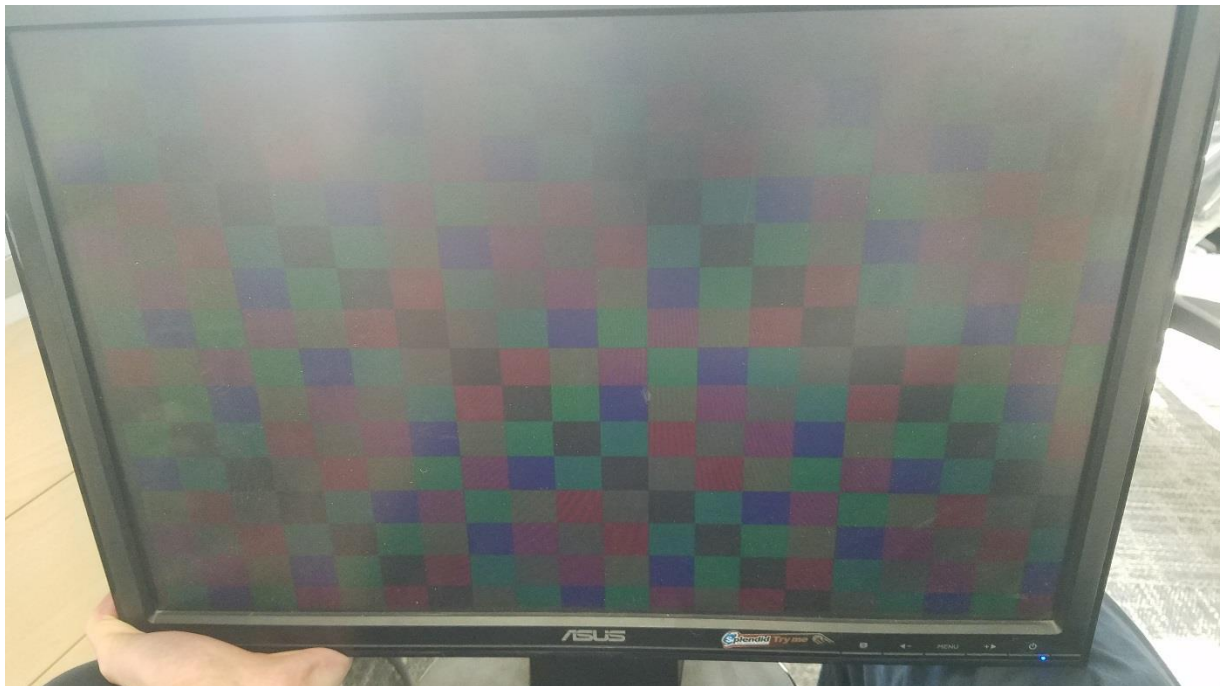
- 5 创建新工程，完成VGA显示控制电路的 VHDL 描述；
- 6 检查程序是否正确，查看 RTL 结构图；
- 7 查看分析VGA显示控制电路的实验结果。

四、实验步骤

- 1、新建一个工程，使用 VHDL 设计一个分频器，使信号分频至25M，刷新频率60hz。分别进行行计数，行同步信号拉低、场计数，场同步信号拉低。
- 2、检查语法错误，综合后，查看 RTL 结构。



3. 分配管脚下载至开发板观察实验结果。并分析实验结果。



五、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、实验结果分析，RTL 结构。实验报告内容为VGA显示控制电路设计。

附件：VGA显示控制电路的VHDL 描述

顶层模块：

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity vga_core is
generic(n:integer:=2);
    Port ( clk : in std_logic;
           reset : in std_logic;
           md : in std_logic_vector(1 downto 0);
           hs : out std_logic;
           vs : out std_logic;
           r : out std_logic;
           g : out std_logic;
           b : out std_logic);
end vga_core;
architecture Behavioral of vga_core is
    signal sysclk : std_logic;
    signal sysclk1 : std_logic;
    signal hsyncb : std_logic;
    signal vsyncb : std_logic;
    signal enable : std_logic;
    signal hloc : std_logic_vector(9 downto 0);
    signal vloc : std_logic_vector(9 downto 0);
    signal rgbx,rgby,rgbp,rgb: std_logic_vector(2 downto 0);
    signal cnt:std_logic_vector(n-1 downto 0);
    component vgasig
        Port (
            clock : in std_logic;
            reset : in std_logic;
            hsyncb : buffer std_logic;
            vsyncb : out std_logic;
            enable : out std_logic;
            Xaddr : out std_logic_vector(9 downto 0);
            Yaddr : out std_logic_vector(9 downto 0)
        );
    end component;
    component colormap
        Port (
            hloc : in std_logic_vector(9 downto 0);
            vloc : in std_logic_vector(9 downto 0);
            rgbx : out std_logic_vector(2 downto 0);
            rgby : out std_logic_vector(2 downto 0)
        );
    end component;
begin
    rgb(2) <= rgbp(2) and enable;
    rgb(1) <= rgbp(1) and enable;
    rgb(0) <= rgbp(0) and enable;
    divclk: process(clk,reset)
    begin
        if reset='0' then
            sysclk <= '0';
        elsif clk'event and clk='1' then
            sysclk <= not sysclk;
        end if;
    end process;
    divclk1: process(sysclk,reset)
    begin
        if reset='0' then
            sysclk1 <= '0';
        elsif sysclk'event and sysclk='1' then
    
```



```
sysclk1 <= not sysclk1;
end if;
end process;
modchoice: process(md, rgbx, rgbx)
begin
    if md="11" then rbgp <= rgbx;
    elsif md="01" then rbgp <= rgbx;
    elsif md="10" then rbgp <= rgbx xor rgbx;
    else rbgp <= "000";
    end if;
end process;
makesig: vgasig Port map(
    clock => sysclk1,
    reset => reset,
    hsyncb => hsyncb,
    vsyncb => vsyncb,
    enable => enable,
    Xaddr => hloc,
    Yaddr => vloc );
makergb: colormap Port map(
    hloc => hloc,
    vloc => vloc,
    rgbx => rgbx,
    rgbx => rgbx );
hs <= hsyncb;
vs <= vsyncb;
r <= rgb(2);
g <= rgb(1);
b <= rgb(0);
end Behavioral;
```

VGA检测模块:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity vgasig is
    Port ( clock : in std_logic;
          reset : in std_logic;
          hsyncb : buffer std_logic;
          vsyncb : out std_logic;
          enable : out std_logic;
          Xaddr : out std_logic_vector(9 downto 0);
          Yaddr : out std_logic_vector(9 downto 0));
end vgasig;
architecture Behavioral of vgasig is
    constant H_PIXELS: INTEGER:=640;
    constant H_FRONT: INTEGER:=16;
    constant H_BACK: INTEGER:=48;
    constant H_SYNCTIME: INTEGER:=96;
    constant H_PERIOD: INTEGER:= H_SYNCTIME + H_PIXELS + H_FRONT + H_BACK;
    constant V_LINES: INTEGER:=480;
```

```
constant V_LINES: INTEGER:=480;
constant V_FRONT: INTEGER:=11;
constant V_BACK: INTEGER:=32;
constant V_SYNC_TIME: INTEGER:=2;
constant V_PERIOD: INTEGER:= V_SYNC_TIME + V_LINES + V_FRONT + V_BACK;
signal hcnt: std_logic_vector(9 downto 0);
signal vcnt: std_logic_vector(9 downto 0);
begin
A: process(clock, reset)
begin
    if reset = '0' then
        hcnt <= (others => '0');
    elsif (clock'event and clock = '1') then
        if hcnt < H_PERIOD then
            hcnt <= hcnt + 1;
        else
            hcnt <= (others => '0');
        end if;
    end if;
end process;
B: process(hsyncb, reset)
begin
    if reset='0' then
        vcnt <= (others => '0');
    elsif (hsyncb'event and hsyncb = '1') then
        if vcnt < V_PERIOD then
            vcnt <= vcnt + 1;
        else
            vcnt <= (others => '0');
        end if;
    end if;
end process;
C: process(clock, reset)
begin
    if reset = '0' then
        hsyncb <= '1';
    elsif (clock'event and clock = '1') then
        if (hcnt >= (H_PIXELS + H_FRONT) and hcnt < (H_PIXELS + H_SYNC_TIME + H_FRONT)) then
            hsyncb <= '0';
        else
            hsyncb <= '1';
        end if;
    end if;
end process;
D: process(hsyncb, reset)
begin
    if reset = '0' then
        vsyncb <= '1';
    elsif (hsyncb'event and hsyncb = '1') then
        if (vcnt >= (V_LINES + V_FRONT) and vcnt < (V_LINES + V_SYNC_TIME + V_FRONT)) then
            vsyncb <= '0';
        else
            vsyncb <= '1';
        end if;
    end if;
end process;
```

```

        end if;
    end process;
E: process (clock)
begin
    if clock'EVENT and clock = '1' then
        if hcnt >= H_PIXELS or vcnt >= V_LINES then
            enable <= '0';
        else
            enable <= '1';
        end if;
    end if;
end process;
H:
Xaddr <= hcnt;
Yaddr <= vcnt;
end Behavioral;

```

色彩模块:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity colormap is
    Port ( hloc : in std_logic_vector(9 downto 0);
          vloc : in std_logic_vector(9 downto 0);
          rgbx : out std_logic_vector(2 downto 0);
          rgby : out std_logic_vector(2 downto 0));
end colormap;
architecture Behavioral of colormap is
begin
    process(hloc, vloc)
    begin
        case hloc(7 downto 5) is
            when "000" => rgbx <= "100";
            when "001" => rgbx <= "110";
            when "010" => rgbx <= "101";
            when "011" => rgbx <= "010";
            when "100" => rgbx <= "001";
            when "101" => rgbx <= "011";
            when "110" => rgbx <= "111";
            when "111" => rgbx <= "000";
            when others => rgbx <= "000";
        end case;
        case vloc(7 downto 5) is
            when "000" => rgby <= "100";
            when "001" => rgby <= "110";
            when "010" => rgby <= "101";
            when "011" => rgby <= "010";
            when "100" => rgby <= "001";
            when "101" => rgby <= "011";
            when "110" => rgby <= "111";
            when "111" => rgby <= "000";
            when others => rgby <= "000";
        end case;
    end process;
end Behavioral;

```

实验八 串口控制器设计

实验类型：

实验学时：

适用专业：

实验房间：

一、实验目的

用状态机实现序列检测器的设计，并对其进行仿真。

二、实验原理

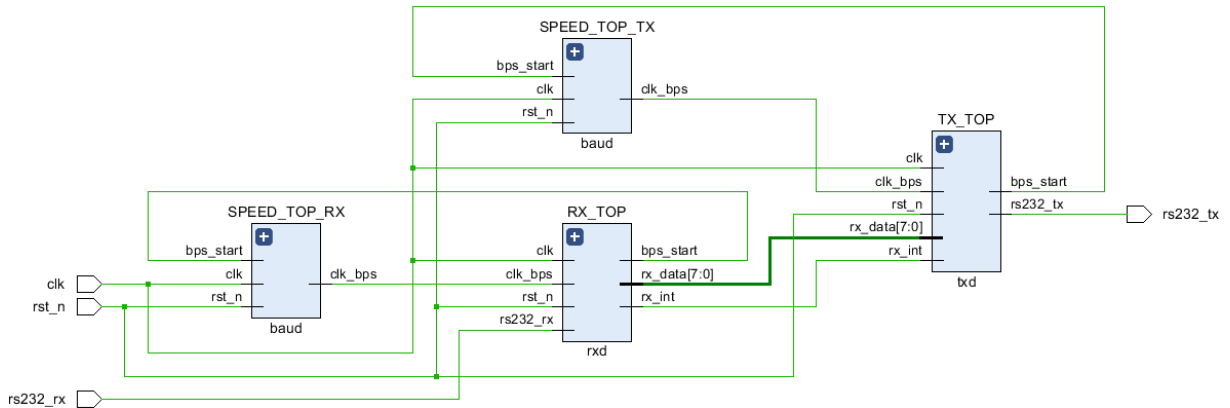
串口通信为单线通信，需要根据传输协议进行传输。一帧数据的开始位拉低电平，表示开始传输数据，随后串行发送8bit数据，随后紧跟1bit高电平停止位，无奇偶校验位。一帧数据传输完毕等待下一帧数据传输。数据传输时间有波特率决定。

三、实验内容

5. 新建一个串口控制器工程；
6. 用 VHDL 语言编写串口控制器源程序；
7. 查看串口控制器的 RTL 结构。
8. 查看分析串口控制器设计的实验结果。

四、实验步骤

- 1、新建一个工程，使用 VHDL 设计一个串口控制器，对总时钟（100M）进行分频，这里分频完成波特率为19200。接收和发送模块的实现依靠有限状态机编写。
- 2、检查语法错误，综合后，查看 RTL 结构。



3. 分配引脚下载至开发板。对实验结果进行分析。



五、实验报告

实验报告包含实验名称、VHDL 源代码(源代码要求手写)、实验结果及分析。

附件：串口控制器的VHDL 描述

1. 顶层文件

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity uart is
    port(clk : in std_logic;
          rst_n: in std_logic;
          rs232_rx: in std_logic;
          rs232_tx: out std_logic
    );
end uart;
architecture behav of uart is
    component rxd port(clk : in std_logic;
                      rst_n: in std_logic;
                      rs232_rx: in std_logic;
                      clk_bps: in std_logic;
                      bps_start:out std_logic;
                      rx_data: out std_logic_vector(7 downto 0);
                      rx_int: out std_logic
    );
end component;
component baud port(clk : in std_logic;
                    rst_n: in std_logic;
                    clk_bps: out std_logic;
                    bps_start:in std_logic
    );
end component;
component txd port(clk : in std_logic;
                  rst_n: in std_logic;
                  rs232_tx: out std_logic;
                  clk_bps: in std_logic;
                  bps_start:out std_logic;
                  rx_data: in std_logic_vector(7 downto 0);
                  rx_int: in std_logic
    );
end component;

    signal bps_start_1:std_logic;
    signal bps_start_2:std_logic;
    signal clk_bps_1:std_logic;
    signal clk_bps_2:std_logic;
    signal rx_data:std_logic_vector(7 downto 0);
    signal rx_int:std_logic;
begin
    RX_TOP: rxd port map(clk=>clk,
                        rst_n=>rst_n,
                        rs232_rx=>rs232_rx,
                        clk_bps=>clk_bps_1,
                        bps_start=>bps_start_1,
                        rx_data=>rx_data,
                        rx_int=>rx_int
    );
    SPEED_TOP_RX: baud port map(clk=>clk,
                                rst_n=>rst_n,
                                clk_bps=>clk_bps_1,
                                bps_start=>bps_start_1
    );
    TX_TOP:txd port map(clk=>clk,
                        rst_n=>rst_n,
                        rs232_tx=>rs232_tx,
                        clk_bps=>clk_bps_2,
                        bps_start=>bps_start_2,
                        rx_data=>rx_data,
                        rx_int=>rx_int
    );
    SPEED_TOP_TX: baud port map(clk=>clk,
                                rst_n=>rst_n,
                                clk_bps=>clk_bps_2,
                                bps_start=>bps_start_2
    );
end behav;

```

2. 发送部分

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity txd is
port (clk : in std_logic;
      rst_n: in std_logic;
      rs232_tx: out std_logic;
      clk_bps: in std_logic;
      bps_start: out std_logic;
      rx_data: in std_logic_vector(7 downto 0);
      rx_int: in std_logic
    );
end txd;
architecture behav of txd is
signal rx_int0: std_logic;
signal rx_int1: std_logic;
signal rx_int2: std_logic;
signal neg_rx_int: std_logic;
signal bps_start_r: std_logic;
signal num: integer;
signal tx_data: std_logic_vector(7 downto 0);
begin
  process (clk, rst_n)
  begin
    if (rst_n='0') then
      rx_int0<='0';
      rx_int1<='0';
      rx_int2<='0';
    else
      if (rising_edge(clk)) then
        rx_int0<=rx_int;
        rx_int1<=rx_int0;
        rx_int2<=rx_int1;
      end if;
    end if;
    neg_rx_int <=not(rx_int1)and (rx_int2);
  end process;

  process (clk, rst_n)
  begin
    if (rst_n='0') then
      bps_start_r<='0';
      tx_data<="00000000";
    else
      if (rising_edge(clk)) then
        if (neg_rx_int='1') then
          bps_start_r<='1';
          tx_data<=rx_data;
        else if ((num=15) and (clk_bps='1')) then
          bps_start_r<='0';
        end if;
      end if;
    end if;
  end process;

  process (clk, rst_n)
  begin
    if (rst_n='0') then
      rs232_tx<='1';
      num<=0;
    else
      if (rising_edge(clk)) then
        if (clk_bps='1') then
          num<=num+1;
          case num is
            when 1=>rs232_tx<='0';
            when 2=>rs232_tx<=tx_data(0); --发送第1bit
            when 3=>rs232_tx<=tx_data(1); --发送第2bit
            when 4=>rs232_tx<=tx_data(2); --发送第3bit
            when 5=>rs232_tx<=tx_data(3); --发送第4bit
            when 6=>rs232_tx<=tx_data(4); --发送第5bit
            when 7=>rs232_tx<=tx_data(5); --发送第6bit
            when 8=>rs232_tx<=tx_data(6); --发送第7bit
            when 9=>rs232_tx<=tx_data(7); --发送第8bit
            when 10=>rs232_tx<='1';
            when 11=>num<=15;
            when others=>null;
          end case;
        end if;
      end if;
    end if;
  end process;
end behav;

```

3. 接收部分

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity rxd is
port(clk : in std_logic;
rst_n: in std_logic;
rs232_rx: in std_logic;
clk_bps: in std_logic;
bps_start: out std_logic;
rx_data: out std_logic_vector(7 downto 0);
rx_int: out std_logic);
end rxd;
architecture behav of rxd is
signal rs232_rx0: std_logic;
signal rs232_rx1: std_logic;
signal rs232_rx2: std_logic;
signal rs232_rx3: std_logic;
signal neg_rs232_rx: std_logic;
signal bps_start_r: std_logic;
signal num: integer;
signal rx_data_r: std_logic_vector(7 downto 0);
begin
process(clk, rst_n)
begin
if (rst_n='0') then
rs232_rx0<='0';
rs232_rx1<='0';
rs232_rx2<='0';
rs232_rx3<='0';
else
if (rising_edge(clk)) then
rs232_rx0<=rs232_rx;
rs232_rx1<=rs232_rx0;
rs232_rx2<=rs232_rx1;
rs232_rx3<=rs232_rx2;
end if;
end if;
neg_rs232_rx<=rs232_rx3 and rs232_rx2
and not(rs232_rx1) and not(rs232_rx0);
end process;
process(clk, rst_n)
begin
if (rst_n='0') then
bps_start_r<='0';
rx_int<='0';
else
if (rising_edge(clk)) then
if (neg_rs232_rx='1') then
bps_start_r<='1';
rx_int<='1';
else if ((num=15) and (clk_bps='1')) then
bps_start_r<='0';
rx_int<='0';
end if;
end if;
end if;
end if;
bps_start<=bps_start_r;
end process;
process(clk, rst_n)
begin
if (rst_n='0') then
rx_data_r<="00000000";
rx_data<="00000000";
num<=0;
else
if (rising_edge(clk)) then
if (clk_bps='1') then
num<=num+1;
case num is
when 1=>rx_data_r(0)<=rs232_rx;--锁存第0bit
when 2=>rx_data_r(1)<=rs232_rx;--锁存第1bit

```



```
when 3=>rx_data_r(2)<=rs232_rx;--锁存第2bit
when 4=>rx_data_r(3)<=rs232_rx;--锁存第3bit
when 5=>rx_data_r(4)<=rs232_rx;--锁存第4bit
when 6=>rx_data_r(5)<=rs232_rx;--锁存第5bit
when 7=>rx_data_r(6)<=rs232_rx;--锁存第6bit
when 8=>rx_data_r(7)<=rs232_rx;--锁存第7bit
when 10=>rx_data<=rx_data_r;
when 11=>num<=15;
when others=>null;
end case;
if(num=15) then
    num<=0;
end if;
end if;
end if;
end if;
end process;
end behav;
```

4. 串口控制器分频（波特率）

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity baud is
    port(clk : in std_logic;
          rst_n: in std_logic;
          clk_bps: out std_logic;
          bps_start: in std_logic
    );
end baud;
architecture behav of baud is
    signal cnt:std_logic_vector(12 downto 0);
    signal clk_bps_r:std_logic;
    constant BPS PARA:integer:=5207;
    constant BPS PARA_2:integer:=2603;
begin
    process(clk,rst_n)
    begin
        if (rst_n='0') then
            cnt<="0000000000000";
        else
            if (rising_edge(clk)) then
                if((cnt=BPS PARA) or (bps_start='0')) then
                    cnt<="0000000000000";
                else
                    cnt<=cnt+'1';
                end if;
            end if;
        end if;
    end process;
    process(clk,rst_n)
    begin
        if (rst_n='0') then
            clk_bps_r<='0';
        else
            if (rising_edge(clk)) then
                if(cnt=BPS PARA_2) then
                    clk_bps_r<='1';
                else
                    clk_bps_r<='0';
                end if;
            end if;
        end if;
        clk_bps<=clk_bps_r;
    end process;
end behav;
```